

**Sari-SaaS: Design and Development of a Secure,
Offline-Capable Unified Commerce Management Platform for
Philippine MSMEs**

A Developmental Research Manuscript

Presented to

[College or Department]

[Institution Name]

In Partial Fulfillment of the Requirements for

[Degree Program]

[Researcher Name]

Adviser: [Research Adviser]

August 2026

Document Status

This manuscript documents an active software research and development project. It reports verified characteristics of the current Sari-SaaS prototype and presents a formal protocol for the field evaluation that should follow. It does not claim that merchant surveys, comparative task experiments, or statistical tests have already been completed. Sections describing future participants, instruments, acceptance thresholds, or hypotheses are therefore written as a proposed evaluation plan. This distinction is necessary to preserve research integrity and prevent technical verification results from being misrepresented as human-subject findings.

Abstract

Micro, small, and medium enterprises (MSMEs) account for 99.63% of registered business establishments in the Philippines, yet many microentrepreneurs still manage online orders, payment evidence, stock, and delivery coordination across separate applications and manual records. This developmental study designs and implements Sari-SaaS, a mobile-first commerce management platform for fragmented merchant workflows, intermittent connectivity, and resource-constrained devices. It applies design science research with Scrum-based iterative development. The TypeScript artifact combines web, mobile, application programming interface, shared-domain, and Supabase PostgreSQL components. Security controls include tenant-scoped row-level security, encrypted integration credentials, short-lived single-use authorization state, signed webhook verification, inactivity-based sign-out, and server-only integration writes. The prototype implements email magic-link authentication, merchant onboarding, an English/Tagalog dashboard, order and inventory views, and secure connection foundations for Facebook, Instagram, TikTok Shop, WooCommerce, and Shopify. Repository linting, static type checking, provider-security tests, database migration, and production builds completed successfully. Full offline mutation synchronization, payment OCR, logistics booking, and transformation of provider events into local orders remain incomplete. A field evaluation with Philippine MSME merchants is therefore proposed using task completion, error rates, the System Usability Scale, Technology Acceptance Model constructs, and ISO/IEC 25010 quality criteria. The principal contribution is a context-sensitive, security-oriented design that translates fragmented social-commerce operations into a testable artifact without overstating prototype maturity.

Keywords: Philippine MSMEs, social commerce, offline-capable systems

Contents

Document Status	i
Abstract	ii
1 Introduction	1
1.1 Background of the Study	1
1.2 Statement of the Problem	2
1.3 Research Questions	3
1.4 Objectives of the Study	3
1.4.1 General Objective	3
1.4.2 Specific Objectives	3
1.5 Significance of the Study	4
1.6 Scope and Delimitations	4
1.7 Definition of Terms	5
2 Review of Related Literature and Systems	7
2.1 MSMEs and Digitalization in the Philippine Context	7
2.2 Social Commerce and Fragmented Operations	8
2.3 Digital Payments and Reconciliation	8
2.4 Offline-Capable and Local-First Design	9
2.5 Technology Acceptance and Information-System Success	9
2.6 Software Quality, Accessibility, and Security	10
2.7 Legal and Regulatory Context	10

2.8	Synthesis and Research Gap	11
3	Methodology	12
3.1	Research Design	12
3.2	Development Method	13
3.3	Conceptual Framework	14
3.4	System Architecture	14
3.5	Data and Security Design	15
3.6	Development and Evaluation Environment	16
3.7	Proposed Participants and Sampling	16
3.8	Proposed Instruments and Measures	17
3.9	Proposed Test Tasks	18
3.10	Data Analysis Plan	19
3.11	Ethical and Privacy Considerations	19
4	Development Results and Current Prototype	20
4.1	Artifact Overview	20
4.2	Channel Integration Result	22
4.3	Database and Tenant Isolation Result	23
4.4	Prototype Verification Result	23
4.5	Discussion	24
5	Summary, Conclusions, and Recommendations	25
5.1	Summary	25
5.2	Conclusions	25
5.3	Recommendations	26
A	Requirements Traceability Matrix	28
B	Proposed Merchant Evaluation Form	30
C	Suggested Acceptance Criteria	32

References

33

List of Tables

3.1	Mapping of Design Science Activities to the Sari-SaaS Study	13
3.2	Proposed Evaluation Measures	17
4.1	Current Prototype Status as of August 2026	20
A.1	Selected Requirements and Evidence	28

List of Figures

3.1	Conceptual framework of the study	14
3.2	Logical architecture of Sari-SaaS	15

Chapter 1

Introduction

1.1 Background of the Study

MSMEs form the numerical foundation of the Philippine business sector. The Department of Trade and Industry reported that 1,236,908 of 1,241,476 registered establishments in 2024 were MSMEs, equivalent to 99.63% of all establishments (Department of Trade and Industry, 2025). Their scale makes improvements in MSME operations relevant not only to individual firms but also to employment, local supply chains, and inclusive economic participation. The Asian Development Bank (ADB) similarly treats MSMEs as a central driver of resilient and inclusive growth across Asia and the Pacific (Asian Development Bank, 2024).

Commercial activity is also becoming increasingly digital. Digital retail payments represented 57.4% of transaction volume and 59.0% of transaction value in the Philippines in 2024 (Bangko Sentral ng Pilipinas, 2025). For small merchants, however, digitalization does not necessarily produce a coherent operating process. A seller may receive inquiries through Facebook or Instagram, accept payment through a mobile wallet, encode the order in a notebook or spreadsheet, update stock separately, and open another application to arrange delivery. Philippine microentrepreneurs interviewed by Romero and Samova (2023) commonly used Facebook for selling and reported benefits from e-commerce, but they also experienced technical problems, slow connectivity, competition, and fraudulent or difficult customer interactions.

This fragmented setting creates an information-management problem. Separate records

increase repeated encoding, delayed responses, missed orders, inventory errors, and weak transaction traceability. The Internet Transactions Act of 2023 also places greater emphasis on trust, merchant accountability, consumer protection, secure internet transactions, and data privacy (Republic of the Philippines, 2023). A useful MSME platform must therefore do more than provide a polished dashboard: it must coordinate existing channels, preserve records, protect personal and commercial data, and remain dependable when connectivity is poor.

Sari-SaaS was conceived as a unified commerce command platform for Philippine MSMEs. The project prioritizes low-cost Android devices, text-first interaction, English and Tagalog localization, offline-capable workflows, and integration with the services merchants already use (Sari-SaaS Development Team, 2026c). Instead of requiring sellers to abandon social networks, wallets, and courier platforms, the system provides a common operational layer for orders, inventory, payments, messages, and delivery activities.

1.2 Statement of the Problem

Philippine MSME merchants who sell through multiple digital channels may lack a single, secure, and connectivity-resilient system for managing the complete transaction lifecycle. The project addresses five related problems:

1. Order information is distributed across social networks, online-store platforms, notes, and spreadsheets.
2. Payment verification and reconciliation often depend on manual review and repeated encoding.
3. Inventory records can become inconsistent when sales are recorded late or separately from stock updates.
4. Courier booking and customer updates require switching between systems.
5. Existing digital workflows may not provide appropriate tenant isolation, auditability, privacy safeguards, accessibility, or graceful operation during unstable connectivity.

The central research problem is therefore how to design, implement, and evaluate a unified commerce management artifact that responds to these operational and technical

conditions without introducing unacceptable security or usability burdens.

1.3 Research Questions

This study is guided by the following questions:

1. How can a mobile-first system consolidate order, inventory, payment, channel, and logistics workflows used by Philippine MSMEs?
2. How can the platform remain responsive and preserve merchant actions during intermittent or unavailable network connectivity?
3. What controls are necessary to protect merchant data, integration credentials, authorization callbacks, and webhook events in a multi-tenant software-as-a-service environment?
4. To what extent does the implemented artifact satisfy selected ISO/IEC 25010 quality characteristics, particularly functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, and flexibility?
5. In a future field evaluation, how usable, useful, and easy to learn is Sari-SaaS for Philippine MSME owners and operators compared with their current fragmented workflow?

1.4 Objectives of the Study

1.4.1 General Objective

The general objective is to design, develop, and evaluate a secure, offline-capable, and localized unified commerce management platform for Philippine MSMEs.

1.4.2 Specific Objectives

The study specifically aims to:

1. model a unified transaction workflow for orders, payments, inventory, sales channels, and delivery;
2. implement a shared web, mobile, API, and database architecture with reusable domain types and validation rules;

3. provide immediate local interaction and a recoverable synchronization design for unstable connectivity;
4. implement tenant isolation, credential encryption, replay-resistant authorization state, signed webhook verification, and secure session handling;
5. support English and Tagalog interfaces with accessible navigation and touch targets;
6. verify the prototype through migration checks, static analysis, automated tests, and production builds; and
7. define a reproducible merchant-centered evaluation protocol for usability, acceptance, task efficiency, reliability, and software quality.

1.5 Significance of the Study

MSME owners and staff. The system may reduce repeated encoding, application switching, missed follow-ups, and uncertainty about stock or payment status. Its localization and low-bandwidth design seek to lower the effort needed to adopt structured digital operations.

Customers. Better transaction records and timely status updates may improve transparency, order accuracy, and access to transaction evidence.

Government and MSME support institutions. The artifact offers a technical reference for programs that promote e-commerce participation, traceability, digital payments, and responsible digitalization.

Software developers and researchers. The project provides an applied case of design science, local-first principles, multi-tenant security, and third-party commerce integration under emerging-market constraints.

Future researchers. The proposed instruments, acceptance thresholds, and traceability matrix can support replication, comparative evaluation, and longitudinal study after deployment.

1.6 Scope and Delimitations

The research artifact includes a Next.js web application, an Expo mobile application scaffold, a Fastify API, shared TypeScript domain packages, and a Supabase PostgreSQL database. The

functional scope covers authentication and merchant onboarding, dashboard indicators, order and inventory views, channel connection metadata, and a secure webhook intake queue. The channel framework supports Facebook, Instagram, TikTok Shop, WooCommerce, and Shopify at different levels of provider readiness.

The study does not claim that all provider approvals, payment gateways, OCR models, logistics APIs, offline synchronization rules, and production observability workflows are complete. Provider webhook events are accepted and deduplicated, but automatic transformation of all events into local orders and stock movements remains future work. The manuscript also does not report field survey outcomes, financial impact, or legal compliance certification. Legal and standards references inform design requirements but do not replace professional legal, privacy, accounting, or security review.

1.7 Definition of Terms

Design science research.

A research paradigm in which a purposeful artifact is created and evaluated to address a relevant organizational problem.

Local-first or offline-capable.

A design approach in which useful work can continue locally and synchronize when communication becomes available (Kleppmann et al., 2019).

MSME.

A micro, small, or medium enterprise as classified by Philippine policy or statistical authorities.

OAuth.

A delegated authorization framework through which a user grants an application limited access without sharing the user's platform password.

Progressive web application (PWA).

A web application enhanced with installability and platform capabilities while retaining browser delivery.

Row-level security (RLS).

Database policy enforcement that limits accessible rows according to the authenticated user and store relationship.

Webhook.

A provider-initiated HTTPS request that communicates an event to another system.

Chapter 2

Review of Related Literature and Systems

2.1 MSMEs and Digitalization in the Philippine Context

The dominance of MSMEs in the Philippine establishment count establishes the relevance of tools designed for small operating teams rather than enterprise departments (Department of Trade and Industry, 2025). The ADB's regional monitoring emphasizes that resilient MSME ecosystems require more than access to technology; they also depend on finance, skills, supportive institutions, market access, and appropriate policy (Asian Development Bank, 2024). This broader view cautions against treating software alone as a complete intervention.

Philippine evidence shows that microentrepreneurs adopt e-commerce pragmatically. Romero and Samova (2023) described bricolage behavior among women microentrepreneurs: available tools and resources were combined to sustain online participation. Such behavior supports an integration-oriented system strategy. A platform is more likely to fit existing practice when it coordinates familiar channels rather than requiring immediate migration to an unfamiliar and comprehensive enterprise system.

At the same time, digitalization can create new burdens if technology is costly, complex, unreliable, or poorly aligned with operating conditions. The research problem is therefore not simply whether merchants use digital platforms, but whether their combined workflow is understandable, efficient, secure, and recoverable.

2.2 Social Commerce and Fragmented Operations

Social commerce combines social interaction with commercial discovery, negotiation, and transaction activity. For microenterprises, social networks lower the cost of market access, but they also mix customer communication with order capture. Conversations, comments, screenshots, and order details may not produce structured records automatically. This creates a need for a unifying layer that can identify the channel while normalizing orders and customer activity into shared domain objects.

The current Sari-SaaS design treats Facebook, Instagram, TikTok Shop, WooCommerce, and Shopify as external systems. It uses delegated authorization or store-generated credentials rather than collecting platform passwords. This approach is consistent with OAuth security guidance that discourages password credential grants and favors authorization-code flows, constrained scopes, redirect validation, and replay protection (Lodderstedt et al., 2025).

2.3 Digital Payments and Reconciliation

The expansion of digital payments establishes both an opportunity and a record management burden. BSP measurements show that a majority of Philippine retail payment volume is now digital (Bangko Sentral ng Pilipinas, 2025). A merchant may still need to match payment evidence to a particular order, detect duplicate references, and preserve a traceable state change. The Sari-SaaS specification therefore proposes two paths: signed gateway webhooks when a direct integration exists and on-device optical character recognition (OCR) for payment screenshots.

Client-side OCR can reduce transmission of payment screenshots, although its accuracy and resource cost must be measured on target devices. Tesseract’s lineage as an adaptable OCR engine is well documented (Smith, 2007), but a general OCR engine does not by itself establish acceptable accuracy for GCash or Maya screenshots. A domain-specific test set, confidence thresholds, manual fallback, and false-match analysis remain necessary.

2.4 Offline-Capable and Local-First Design

Local-first research argues that a local copy can support immediate access and work during disconnection while synchronization restores collaboration later (Kleppmann et al., 2019). These principles are relevant to merchants who use prepaid data or encounter unstable mobile signals. A local-first design can reduce blocking network round trips, but it also introduces conflict, ordering, durability, and reconciliation problems.

The project currently proposes IndexedDB for web storage, SQLite for mobile storage, and a mutation queue that synchronizes deltas. Last-write-wins may be acceptable for low-risk profile fields but is insufficient for all inventory and payment conflicts. Stock quantities, payment state, and fulfillment events require domain-specific idempotency keys, monotonic state transitions, and auditable conflict handling. The literature therefore supports offline work as a design goal while also demonstrating why synchronization must be evaluated as a subsystem rather than treated as a browser caching feature.

2.5 Technology Acceptance and Information-System Success

The Technology Acceptance Model identifies perceived usefulness and perceived ease of use as determinants of user acceptance (Davis, 1989). This model is appropriate to the target population because Sari-SaaS competes with simple but familiar practices such as notebooks and direct use of social media. An artifact can be technically sophisticated and still fail if merchants do not believe it saves time or if initial setup appears difficult.

The DeLone and McLean model connects system quality, information quality, service quality, use, satisfaction, and net benefits (DeLone & McLean, 2003). Together, these frameworks justify evaluating more than interface preference. The proposed study measures task success and errors, perceived usefulness and ease, satisfaction, reliability, and the quality of operational information.

2.6 Software Quality, Accessibility, and Security

ISO/IEC 25010:2023 defines a product quality model with nine characteristics that can guide requirements and evaluation throughout the software lifecycle (International Organization for Standardization, 2023). Sari-SaaS prioritizes functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, and flexibility. Safety is considered where incorrect status or inventory information could influence merchant decisions.

WCAG 2.2 organizes accessibility around perceivable, operable, understandable, and robust content and provides testable success criteria for web interfaces (World Wide Web Consortium, 2024). The project's text-first controls, visible labels, responsive layout, high contrast, and large touch targets are consistent with that direction, but formal conformance must still be audited across complete pages and interaction states.

Security is treated as an architectural quality rather than a final checklist. The Data Privacy Act regulates the processing lifecycle of personal data and establishes accountability obligations (Republic of the Philippines, 2012). OWASP ASVS provides verifiable web-application security requirements (OWASP Foundation, 2025), while NIST digital identity guidance informs authentication and session controls (National Institute of Standards and Technology, 2025). For external channels, OAuth best current practice supports authorization-code exchange, state validation, least privilege, TLS, and protection against token replay and redirect attacks (Lodderstedt et al., 2025).

2.7 Legal and Regulatory Context

The Internet Transactions Act applies to covered business-to-business and business-to-consumer transactions involving the Philippine market. It promotes trust, consumer protection, secure transactions, data privacy, transparent offers, and identifiable parties (Republic of the Philippines, 2023). The law supports the project's emphasis on merchant profiles, transaction records, receipt-ready data, and traceability. Nevertheless, software features are not equivalent to certification. Production deployment requires review of the Act's implementing rules, consumer remedies, registration obligations, tax and invoicing requirements, retention policies, and provider terms.

2.8 Synthesis and Research Gap

The literature supports digital commerce participation, offline-capable interaction, usable systems, and security controls, but these concerns are often studied or implemented separately. Philippine microentrepreneurs commonly assemble multiple general-purpose tools, while enterprise commerce systems may assume stable connectivity, dedicated staff, or formal online-store operations. The research gap addressed by Sari-SaaS is the design and evaluation of a single localized artifact that coordinates social channels, online stores, payment evidence, stock, and delivery while explicitly accounting for low-resource devices, connectivity interruptions, multi-tenant security, and merchant acceptance.

Chapter 3

Methodology

3.1 Research Design

The study uses design science research because its primary output is a purposeful information-system artifact. Hevner et al. (2004) describe design science as the construction and evaluation of artifacts that extend human and organizational capabilities. The process follows the six activities proposed by Peffers et al. (2007): problem identification and motivation, definition of solution objectives, design and development, demonstration, evaluation, and communication.

The study is also developmental and mixed-method in its proposed evaluation. Technical verification produces quantitative measures such as build success, latency, task completion, error counts, and synchronization outcomes. Merchant testing adds scaled acceptance responses and qualitative observations or interviews. No causal claim will be made without an appropriate comparative design and sufficient data.

Table 3.1: Mapping of Design Science Activities to the Sari-SaaS Study

DSR activity	Application in this study
Problem identification	Analyze fragmented merchant workflows, connectivity constraints, transaction traceability, and security risks.
Solution objectives	Define a localized, mobile-first, integrated, secure, and offline-capable commerce platform.
Design and development	Build the monorepo, shared domain model, web/mobile clients, API, database policies, and provider integration layer.
Demonstration	Execute representative onboarding, order, inventory, and channel connection workflows in a controlled environment.
Evaluation	Perform technical verification and a future merchant-centered usability and acceptance study.
Communication	Produce the software repository, architecture decisions, technical guides, and this research manuscript.

3.2 Development Method

Development is organized using Scrum with a two-week sprint model and a prioritized product backlog (Sari-SaaS Development Team, 2026a). Scrum is an iterative framework based on transparency, inspection, and adaptation (Schwaber & Sutherland, 2020). The backlog groups work into authentication, offline data, messaging, payment reconciliation, orders, inventory, dashboard, integrations, logistics, compliance, billing, and analytics.

Each increment should satisfy a definition of done that includes acceptance criteria, automated checks, target-device review, security impact review, and documentation. Features involving external providers are not considered complete merely because a UI card exists; authorization, callback validation, credential protection, webhook authenticity, error recovery, and event processing must also be demonstrated.

3.3 Conceptual Framework

The conceptual framework combines an input-process-output structure with acceptance and quality outcomes. Figure 3.1 shows how merchant needs, technical constraints, regulatory duties, and design knowledge enter an iterative design-science process. The output is the Sari-SaaS artifact, while evaluation considers software quality, user acceptance, and operational effect.

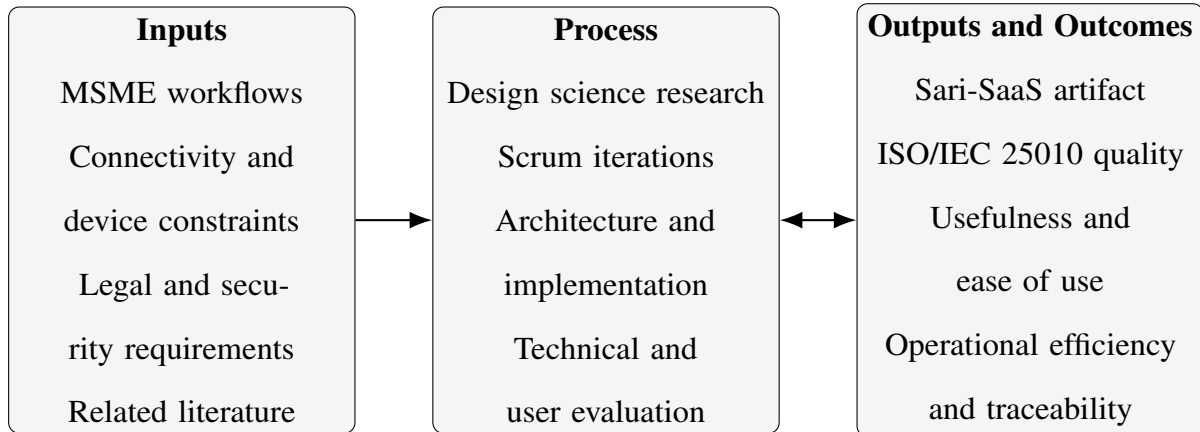


Figure 3.1: Conceptual framework of the study

3.4 System Architecture

The artifact uses a monorepo so web, mobile, API, database, and shared packages can evolve under common type definitions. Figure 3.2 presents the logical architecture.

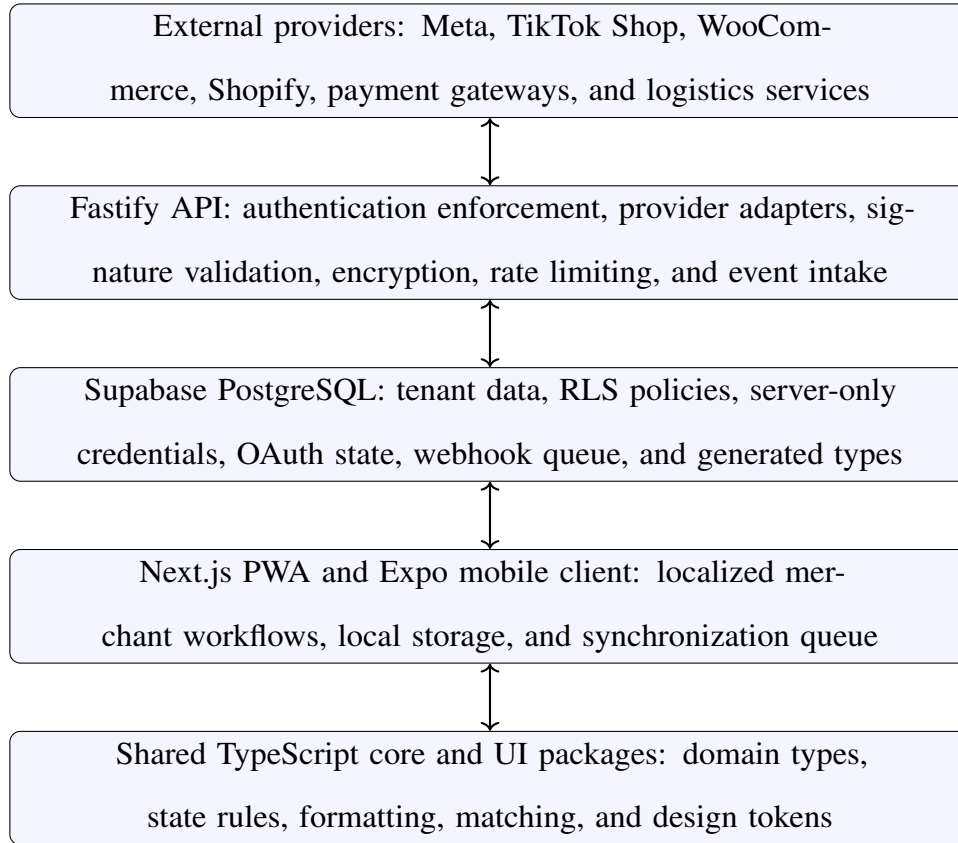


Figure 3.2: Logical architecture of Sari-SaaS

3.5 Data and Security Design

The primary entities are users, stores, orders, payments, messages, inventory, shipments, integration metadata, encrypted integration credentials, webhook events, and synchronization records. Store ownership forms the tenant boundary. Browser clients may read or modify only rows allowed by RLS policies, while credential and authorization tables are restricted to the server service role.

The security design applies the following controls:

1. short-lived email sign-in links and inactivity-based session timeout;
2. per-store authorization and database RLS;
3. AES-256-GCM encryption for third-party credentials with contextual authenticated data;
4. hashed, short-lived, single-use authorization state;
5. server-side token exchange and absence of provider secrets in browser environment variables;

6. provider-specific HMAC verification over the unchanged webhook body;
7. deduplication by provider and external event identifier;
8. HTTPS requirements, strict Shopify domain validation, and public-address validation with DNS pinning for merchant-supplied WooCommerce hosts;
9. API rate limiting and redaction of authorization-bearing request data; and
10. automatic credential removal from local storage when an integration is disconnected or a Shopify uninstall event is received.

These controls are informed by ASVS and OAuth guidance but require independent penetration testing before production assurance can be claimed (Lodderstedt et al., 2025; OWASP Foundation, 2025).

3.6 Development and Evaluation Environment

The reference environment uses Node.js 20 or later, pnpm workspaces, TypeScript, Next.js, Expo, Fastify, and Supabase PostgreSQL. The repository defines lint, type-check, test, build, migration, and type-generation commands. Testing should cover a desktop browser, a low-cost Android device with 3 GB of memory, and network profiles representing stable broadband, weak mobile data, intermittent connection, and complete disconnection.

3.7 Proposed Participants and Sampling

The field evaluation should recruit Philippine MSME owners or staff who directly perform at least two of the following: receive online orders, confirm digital payments, update inventory, communicate with customers, or arrange delivery. Purposive sampling is appropriate for initial formative evaluation because the artifact targets a specific operational role. A proposed initial sample is 30 to 50 participants across micro and small enterprises, with diversity in age, device capability, location, language preference, and primary sales channel.

The final sample size should be confirmed through the institution's research protocol and the intended statistical analysis. A small formative usability round may precede the larger evaluation to identify severe interface defects. Participants must provide informed consent,

and no provider password, live payment credential, or unnecessary customer data should be collected.

3.8 Proposed Instruments and Measures

Table 3.2: Proposed Evaluation Measures

Construct	Measure	Purpose
Task effectiveness	Completion, assistance, critical errors, and data accuracy	Determine whether merchants can complete core workflows correctly.
Task efficiency	Completion time, taps/clicks, repeated encoding, and app switches	Compare operational effort with the current workflow.
Usability	Ten-item System Usability Scale (SUS)	Produce a standardized summary usability score (Bangor et al., 2008; Brooke, 1996).
Acceptance	Perceived usefulness, perceived ease of use, and behavioral intention items	Evaluate adoption-related perceptions based on TAM (Davis, 1989).
Software quality	Expert rubric derived from ISO/IEC 25010	Assess product quality characteristics and documented evidence (International Organization for Standardization, 2023).

Construct	Measure	Purpose
Accessibility	WCAG 2.2 automated and manual checks	Examine keyboard access, labels, contrast, reflow, focus, and target sizing (World Wide Web Consortium, 2024).
Reliability	Offline writes, reconnect synchronization, duplicate events, retries, and recovery	Measure continuity and integrity under network change.
Security	Authorization, RLS, tenant isolation, secret exposure, signature, replay, and SSRF test cases	Verify selected controls against documented requirements (OWASP Foundation, 2025).

3.9 Proposed Test Tasks

Participants should complete representative tasks using synthetic data:

1. sign in, complete store onboarding, and switch the interface language;
2. record a walk-in or direct sale and review its payment state;
3. add or inspect inventory and identify a low-stock product;
4. locate pending payments and return to the dashboard;
5. connect a sandbox or development sales channel where provider access is available;
6. continue a supported action during simulated disconnection and verify its state after reconnection; and
7. sign out or allow the inactivity timeout to protect the session.

No task should use a participant's production credentials or real customer records. Provider connection testing should use development shops, test users, and revocable credentials.

3.10 Data Analysis Plan

Descriptive statistics should summarize task completion, error rates, times, SUS scores, quality ratings, and acceptance constructs. Internal consistency of multi-item scales should be assessed before computing composite scores. If the same participants perform a baseline workflow and the Sari-SaaS workflow, paired analysis may be used after checking distributional assumptions. Qualitative observations should be coded into recurring themes such as navigation, terminology, trust, connectivity, setup burden, and perceived usefulness.

The following hypotheses may be tested after sufficient field data exist:

- H1.** Perceived ease of use is positively associated with behavioral intention to use Sari-SaaS.
- H2.** Perceived usefulness is positively associated with behavioral intention to use Sari-SaaS.
- H3.** Median task completion time is lower with Sari-SaaS than with the participant's current fragmented workflow for matched tasks.
- H4.** Transaction-record errors are lower with Sari-SaaS than with the current workflow for matched tasks.

3.11 Ethical and Privacy Considerations

The evaluation should undergo any ethics review required by the institution. Participation must be voluntary, and participants may withdraw without penalty. The research dataset should use participant codes rather than direct identifiers, collect only necessary variables, define retention and deletion periods, and restrict access to the research team. Screenshots, access tokens, customer messages, and transaction references must be synthetic or redacted. These procedures support the proportionality, transparency, and accountability principles reflected in Philippine data privacy regulation (Republic of the Philippines, 2012).

Chapter 4

Development Results and Current Prototype

4.1 Artifact Overview

The current artifact is a working monorepo rather than a paper-only design. Its web surface contains authenticated onboarding, a merchant dashboard, order and inventory pages, language selection, session timeout handling, and sales-channel management. The API includes protected route modules and a server-only channel credential vault. The database contains sequential migrations, ownership relationships, RLS policies, integration metadata, authorization state, encrypted credential storage, and a deduplicated webhook queue (Sari-SaaS Development Team, 2026b).

Table 4.1: Current Prototype Status as of August 2026

Capability	Status	Evidence and limitation
Email authentication	Implemented	Magic-link interface, 15-minute email-link configuration, localized messages, and 30-minute inactivity sign-out.

Capability	Status	Evidence and limitation
Store onboarding	Implemented	Authenticated profile and atomic store-creation workflow with ownership checks.
Dashboard	Implemented foundation	Today's sales, pending payments, low stock, new sale, screenshot scan, rider booking, and channels navigation are visible; some actions remain incomplete.
Orders and inventory	Implemented foundation	Store-scoped list views and navigation are present; full operational CRUD and offline conflict behavior require further testing.
Localization	Implemented foundation	English and Tagalog interface selection is available for the implemented web screens.
Facebook, Instagram, TikTok Shop	Secure connection foundation	OAuth or provider authorization, account selection, encrypted credentials, and signed webhook intake exist; production availability depends on provider approval and event processors.
WooCommerce	Secure connection foundation	Per-store HTTPS authorization, encrypted REST keys, public-host validation, pinned DNS resolution, and signed webhook intake are implemented.
Shopify	Secure connection foundation	Verified OAuth callback, encrypted offline token, shop validation, webhook registration, signature verification, and uninstall handling are implemented.

Capability	Status	Evidence and limitation
Offline-first synchronization	Partial/scaffold	Data model and sync queue exist, but complete IndexedDB-first behavior and domain-specific conflict resolution are not yet demonstrated end-to-end.
Payment OCR	Scaffold	Matching utilities and planned client-side OCR path exist; device accuracy and latency are not yet established.
Logistics orchestration	Scaffold	Provider types and API route boundaries exist; production quote, booking, and tracking integrations remain pending.
Compliance output	Scaffold	Merchant fields and receipt-oriented domain logic exist; legal validation and production e-invoicing are outside the current prototype.

4.2 Channel Integration Result

The channel layer separates browser-readable integration status from encrypted credentials and raw provider events. The browser requests a connection from the authenticated API. The API creates a random state value, stores only its hash, sets a ten-minute expiration, and encrypts any provider-specific context. After authorization, credentials are exchanged or received only by the server and are encrypted before database storage.

Meta and TikTok Shop may return multiple pages or shops, in which case the merchant selects the intended account without receiving an access token in the browser. WooCommerce authorizes against the merchant's own WordPress domain. Because this domain is user-supplied, the server requires HTTPS, rejects local and private addresses, validates DNS results, pins the

outbound connection to a validated public address, rejects redirects, and preserves certificate verification for the requested hostname. Shopify accepts only permanent `myshopify.com` domains and verifies callback HMAC, state, and shop identity before token exchange.

Webhook handlers verify provider-specific signatures against the raw request body before parsing or storing an event. Events are linked to a connected store integration and deduplicated by provider and external event identifier. This is a secure intake result, not yet a completed order synchronization result.

4.3 Database and Tenant Isolation Result

The database schema links merchant records to a store identifier and enables RLS on merchant-controlled tables. Ownership policies compare the authenticated user with the store owner. Integration credential, OAuth state, and webhook tables revoke access from anonymous and authenticated browser roles, leaving them accessible to the server service role. Browser writes to integration metadata were also revoked after removal of the former manual-channel mode.

This policy structure reduces accidental cross-store access, but automated negative tests should be expanded. Each table and database function should be tested with an owner, a different authenticated owner, an anonymous client, and the service role. The service key must remain outside all public web bundles.

4.4 Prototype Verification Result

The repository was verified on August 10, 2026. The following commands completed successfully: workspace linting, workspace TypeScript checking, the available automated test suite, and production builds. Four provider-security tests passed: Shopify domain restriction, WooCommerce private-address rejection, Shopify raw body HMAC verification, and WooCommerce per-store HMAC verification. Database migration 011 was applied to the linked Supabase project and TypeScript database definitions were regenerated from the resulting schema.

These outcomes provide reproducible evidence of build integrity and selected security behaviors. They do not establish complete correctness, penetration resistance, accessibility

conformance, field usability, OCR accuracy, sync reliability, or business benefit. Automated test coverage remains narrow relative to the artifact’s intended scope.

4.5 Discussion

The current result demonstrates that the project architecture can support a secure integration boundary without exposing provider credentials to the browser. Shared TypeScript types and generated database types reduce contract drift between modules. The sequential migration history supports reproducible database evolution, and RLS places tenant isolation near the data rather than relying solely on application filters.

The prototype also reveals the difference between visible completeness and operational completeness. A channel card may connect successfully while the event-to-order processor is still absent. A sync queue may exist while conflict rules remain untested. An OCR library may run while false matches remain unsafe. For this reason, the status table uses “foundation”, “partial”, and “scaffold” deliberately. This conservative reporting makes the next research iteration measurable.

Chapter 5

Summary, Conclusions, and Recommendations

5.1 Summary

This research addresses fragmented digital-commerce operations among Philippine MSMEs by designing and developing Sari-SaaS, a localized and offline-capable commerce management platform. The study is grounded in the economic importance of MSMEs, increasing digital-payment use, observed e-commerce participation among Filipino microentrepreneurs, and stronger legal expectations for trusted internet transactions.

The artifact follows design science research and Scrum-based iterative development. Its architecture combines web and mobile clients, a typed API, shared domain packages, and a PostgreSQL database with tenant policies. Current implementation results include secure authentication and onboarding foundations, dashboard and operational views, localization, and a hardened sales-channel authorization and webhook layer. Technical verification passed, but major modules and field evaluation remain unfinished.

5.2 Conclusions

The current Sari-SaaS prototype provides credible evidence that a unified commerce platform can be structured around the constraints of Philippine MSMEs without placing third-party

credentials in the browser or making every workflow depend on continuous connectivity. The use of shared types, sequential migrations, RLS, encrypted credentials, authorization state, and signed webhook verification gives the project a defensible technical foundation.

The research questions concerning merchant usability, acceptance, task efficiency, and operational benefit cannot yet be answered empirically. The appropriate conclusion is therefore conditional: the artifact is sufficiently developed for continued controlled testing, but not sufficiently evaluated for claims of field effectiveness or production readiness. Its research value lies in the contextual design, implemented security boundary, transparent maturity assessment, and evaluation plan.

5.3 Recommendations

1. Implement an idempotent event processor that maps approved provider events to canonical orders, products, and inventory movements with retries and a dead-letter workflow.
2. Complete IndexedDB-first writes and define conflict policies by entity; do not use one last-write-wins rule for payments, stock, and fulfillment.
3. Build payment screenshot fixtures and measure OCR precision, recall, false matches, confidence calibration, memory use, and processing time on the target Android device.
4. Expand automated testing to API authorization, RLS isolation, callback replay, token expiry, webhook duplication, malformed events, session timeout, and synchronization recovery.
5. Conduct an accessibility audit against WCAG 2.2 Level AA before making a conformance claim.
6. Obtain provider development credentials and app approvals, then run end-to-end sandbox tests before production merchant onboarding.
7. Perform independent security review and penetration testing, including tenant isolation, secret management, SSRF, OAuth mix-up, webhook replay, and dependency risk.
8. Submit the merchant evaluation protocol for institutional ethics review, conduct formative testing, revise the artifact, and only then execute the larger comparative study.

9. Review Internet Transactions Act implementing rules, Data Privacy Act obligations, BIR invoicing requirements, retention schedules, and provider terms with qualified professionals.
10. Revisit cost, pricing, and merchant support assumptions with real usage data rather than relying on projected adoption alone.

Appendix A

Requirements Traceability Matrix

Table A.1: Selected Requirements and Evidence

ID	Requirement	Current evidence	Next validation
AUTH-01	Passwordless merchant sign-in	Email magic-link screen and Supabase authentication	Expiry, replay, same-browser, and recovery test matrix
AUTH-02	Automatic inactivity sign-out	Thirty-minute session timeout component	Multi-tab and background-device tests
TENANT-01	Store-level isolation	Ownership RLS policies and server-only tables	Automated cross-tenant negative tests
CHAN-01	Secure channel authorization	Hashed state, encrypted context, callback verification, credential vault	Provider sandbox and replay tests

ID	Requirement	Current evidence	Next validation
CHAN-02	Authentic event intake	Raw-body HMAC checks and deduplicated event queue	Fixture coverage for every event topic
OFF-01	Continue work while offline	Sync schema and architecture decision record	End-to-end local write and reconnect tests
PAY-01	Reconcile payment evidence	Domain matching utilities and OCR design	Labeled screenshot corpus and device benchmark
INV-01	Identify low stock	Inventory schema and low-stock page	Accuracy and merchant task test
LOC-01	English and Tagalog UI	Language context and translated implemented screens	Translation review with target users
ACC-01	Accessible interaction	Text labels, responsive cards, visible back navigation, touch sizing	WCAG 2.2 AA audit
QUAL-01	Reproducible build quality	Lint, type-check, tests, and production build complete	CI enforcement and coverage thresholds

Appendix B

Proposed Merchant Evaluation Form

Participant Profile

Collect only variables necessary for analysis: role, enterprise size, years in operation, primary sales channels, typical daily order range, device class, usual connectivity, and preferred language. Avoid collecting customer names, wallet numbers, passwords, or live transaction references.

Post-Task Questions

After each task, ask the participant to rate task difficulty on a seven-point scale and briefly explain any confusion, unexpected outcome, or missing information. The observer records completion, assistance, critical errors, noncritical errors, time, and notable navigation behavior.

Post-Test Scales

Administer the standard SUS wording without unauthorized modification, followed by clearly labeled TAM-based perceived usefulness, perceived ease of use, and behavioral intention items. Any translated instrument should undergo forward translation, independent back-translation, pilot review, and reliability assessment before inferential use.

Open-Ended Questions

1. Which part of your current selling workflow is most difficult?
2. Which Sari-SaaS feature was most useful and why?
3. Which step was confusing or required help?
4. What would prevent you from using this system regularly?
5. What information would make you trust the system with store records?
6. What should work even when mobile data is unavailable?

Appendix C

Suggested Acceptance Criteria

These thresholds are proposed project gates rather than reported outcomes:

- at least 85% unassisted completion for each critical merchant task;
- zero unrecovered data loss in planned offline/reconnect scenarios;
- 100% rejection of invalid tenant access, callback state, and webhook signatures in the defined security suite;
- no provider credential in public bundles, browser storage, logs, or browser-readable database responses;
- a median SUS score at or above 68 as an initial benchmark, interpreted with the score distribution and participant context (Bangor et al., 2008);
- WCAG 2.2 Level AA findings documented and severe barriers remediated;
- API latency and client interaction budgets measured under declared hardware and network profiles; and
- every production integration supported by a runbook for credential rotation, provider outage, webhook retry, disconnect, and incident response.

References

- Asian Development Bank. (2024). *Asia small and medium-sized enterprise monitor 2024: Designing an msme ecosystem for resilient growth in asia and the pacific*. Asian Development Bank. <https://doi.org/10.22617/SGP240536-2>
- Bangko Sentral ng Pilipinas. (2025). *2024 report on e-payments measurement: From surge to stability*. Bangko Sentral ng Pilipinas. Retrieved August 10, 2026, from https://www.bsp.gov.ph/PaymentAndSettlement/2024_Report_on_E-payments_Measurement.pdf
- Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the system usability scale. *International Journal of Human–Computer Interaction*, 24(6), 574–594. <https://doi.org/10.1080/10447310802205776>
- Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, I. L. McClelland, & B. Weerdmeester (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319–340. <https://doi.org/10.2307/249008>
- DeLone, W. H., & McLean, E. R. (2003). The DeLone and McLean model of information systems success: A ten-year update. *Journal of Management Information Systems*, 19(4), 9–30. <https://doi.org/10.1080/07421222.2003.11045748>
- Department of Trade and Industry. (2025). *2024 philippine msme statistics*. Retrieved August 10, 2026, from <https://www.dti.gov.ph/dti-knowledge-hub/dti-statistics/>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>

- International Organization for Standardization. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (SQuaRE)—product quality model* (ISO/IEC 25010:2023). International Organization for Standardization. <https://www.iso.org/standard/78176.html>
- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 154–178. <https://doi.org/10.1145/3359591.3359737>
- Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025). *Best current practice for oauth 2.0 security* (RFC 9700). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9700>
- National Institute of Standards and Technology. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST SP 800-63B-4). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-63B-4>
- OWASP Foundation. (2025). *Application security verification standard 5.0.0*. OWASP Foundation. Retrieved August 10, 2026, from <https://owasp.org/www-project-application-security-verification-standard/>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Republic Act No. 10173: Data Privacy Act of 2012 (2012, August 15). Retrieved August 10, 2026, from <https://privacy.gov.ph/data-privacy-act/>
- Republic Act No. 11967: Internet Transactions Act of 2023 (2023, December 5). Retrieved August 10, 2026, from https://lawphil.net/statutes/repacts/ra2023/ra_11967_2023.html
- Romero, R. Y., & Samova, H. (2023). *Understanding the e-commerce participation of filipino women microentrepreneurs*. AIM Rizalino S. Navarro Policy Center for Competitiveness. Retrieved August 10, 2026, from <https://serp-p.pids.gov.ph/publication/public/view?slug=understanding-the-e-commerce-participation-of-filipino-women-microentrepreneurs>

- Sari-SaaS Development Team. (2026a, May). *Sari-saas scrum agile planning and user stories: Sprint backlog, version 1.0* [Unpublished internal planning document].
- Sari-SaaS Development Team. (2026b, August 10). *Sari-saas unified msme command engine* (Version Prototype) [Unpublished software repository].
- Sari-SaaS Development Team. (2026c, May). *Sari-saas unified msme command engine: Project specification, version 1.1* [Unpublished internal project specification].
- Schwaber, K., & Sutherland, J. (2020). *The scrum guide: The definitive guide to scrum: The rules of the game*. Scrum Guides. Retrieved August 10, 2026, from <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>
- Smith, R. (2007). An overview of the Tesseract ocr engine. *Ninth International Conference on Document Analysis and Recognition*, 2, 629–633. <https://doi.org/10.1109/ICDAR.2007.4376991>
- World Wide Web Consortium. (2024, December 12). *Web content accessibility guidelines (WCAG) 2.2*. World Wide Web Consortium. Retrieved August 10, 2026, from <https://www.w3.org/TR/WCAG22/>